触摸显示

触摸显示
- 例程实验效果简介
- 例程代码
  - 完整代码
  - 代码结构
- TOUCH 相关文档
  - 1. 概述
  - 2. API 介绍
    - 构造函数
    - `read` 方法
    - `deinit` 方法
  - 3. TOUCH_INFO 类
  - 4. 常量
    - 4.1 触摸事件
    - 4.2 坐标旋转
    - 4.3 触摸类型

例程实验效果简介

> 本节我们演示K230触摸屏的使用，需要选配屏幕

例程代码位于 【源码汇总 / 02.Basic / 16.touch.py】

我们用CanMV IDE打开例程代码，将K230用USB连接到电脑上

点击左下角运行按钮即可开始运行程序

初始界面是白色的

我们用手指在屏幕上移动，可以看到随着手指的移动，屏幕上绘制出黑色的线条

例程代码

完整代码

```
# Import required modules
# 导入所需的模块
import time, os, urandom, sys

# Import display and media related modules
# 导入显示和媒体相关模块
from media.display import *
from media.media import *

# Import touch sensor module
# 导入触摸传感器模块
from machine import TOUCH

# Initialize touch sensor on pin 0
```

```
# 在引脚0上初始化触摸传感器
tp = TOUCH(0)

# Define display resolution constants
# 定义显示分辨率常量
DISPLAY_WIDTH = 640
DISPLAY_HEIGHT = 480

def display_test():
    """
    Function to test display and touch functionality
    测试显示和触摸功能的函数
    """
    print("display and touch test")

    # Create main background image with white color
    # 创建白色背景的主图像
    img = image.Image(DISPLAY_WIDTH, DISPLAY_HEIGHT, image.ARGB8888)
    img.clear()
    img.draw_rectangle(0, 0, DISPLAY_WIDTH, DISPLAY_HEIGHT,color=
(255,255,255),fill=True)

    # Create secondary image for drawing
    # 创建用于绘画的次要图像
    img2 = image.Image(DISPLAY_WIDTH, DISPLAY_HEIGHT, image.ARGB8888)
    img2.clear()

    # Initialize display with ST7701 driver
    # 使用ST7701驱动初始化显示器
    Display.init(Display.ST7701, width = DISPLAY_WIDTH, height = DISPLAY_HEIGHT,
to_ide = True)
    # Initialize media manager
    # 初始化媒体管理器
    MediaManager.init()

    try:
        # Variables to store previous touch coordinates
        # 存储上一次触摸坐标的变量
        last_x = None
        last_y = None
        while True:
            os.exitpoint()
            # Read touch point data
            # 读取触摸点数据
            point = tp.read(1)

            if len(point):
                print(point)
                pt = point[0]
                # Handle touch events (down or move)
                # 处理触摸事件（按下或移动）
                if pt.event == 0 or pt.event == TOUCH.EVENT_DOWN or pt.event ==
TOUCH.EVENT_MOVE:
                    if((last_x is not None) and (last_y is not None) and
pt.event is not 2):
                        # Draw line between previous and current touch points
                        # 在上一个触摸点和当前触摸点之间画线
```

```
                        img2.draw_line(last_x,last_y,pt.x, pt.y, color=(0,0,0),
thickness = 5)
                        Display.show_image(img2, layer = Display.LAYER_OSD2,
alpha = 128)
                    last_x = pt.x
                    last_y = pt.y
            # Update display with background image
            # 更新显示背景图像
            Display.show_image(img)

            time.sleep(0.05)
    except KeyboardInterrupt as e:
        print("user stop: ", e)
    except BaseException as e:
        print(f"Exception {e}")

    # Cleanup and deinitialize display
    # 清理并反初始化显示器
    Display.deinit()
    os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
    time.sleep_ms(100)
    # Release media resources
    # 释放媒体资源
    MediaManager.deinit()

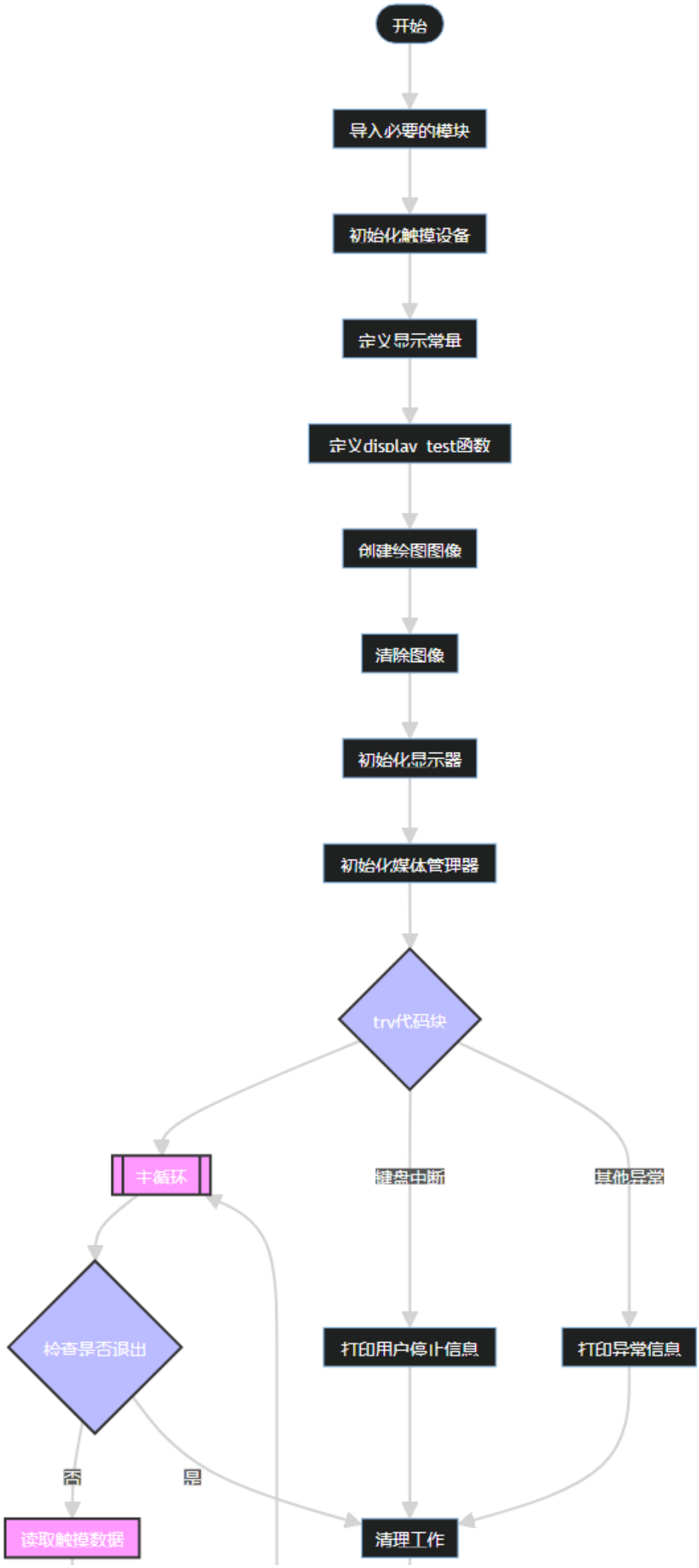
if __name__ == "__main__":
    # Enable exit points and run display test
    # 启用退出点并运行显示测试
    os.exitpoint(os.EXITPOINT_ENABLE)
    display_test()
```

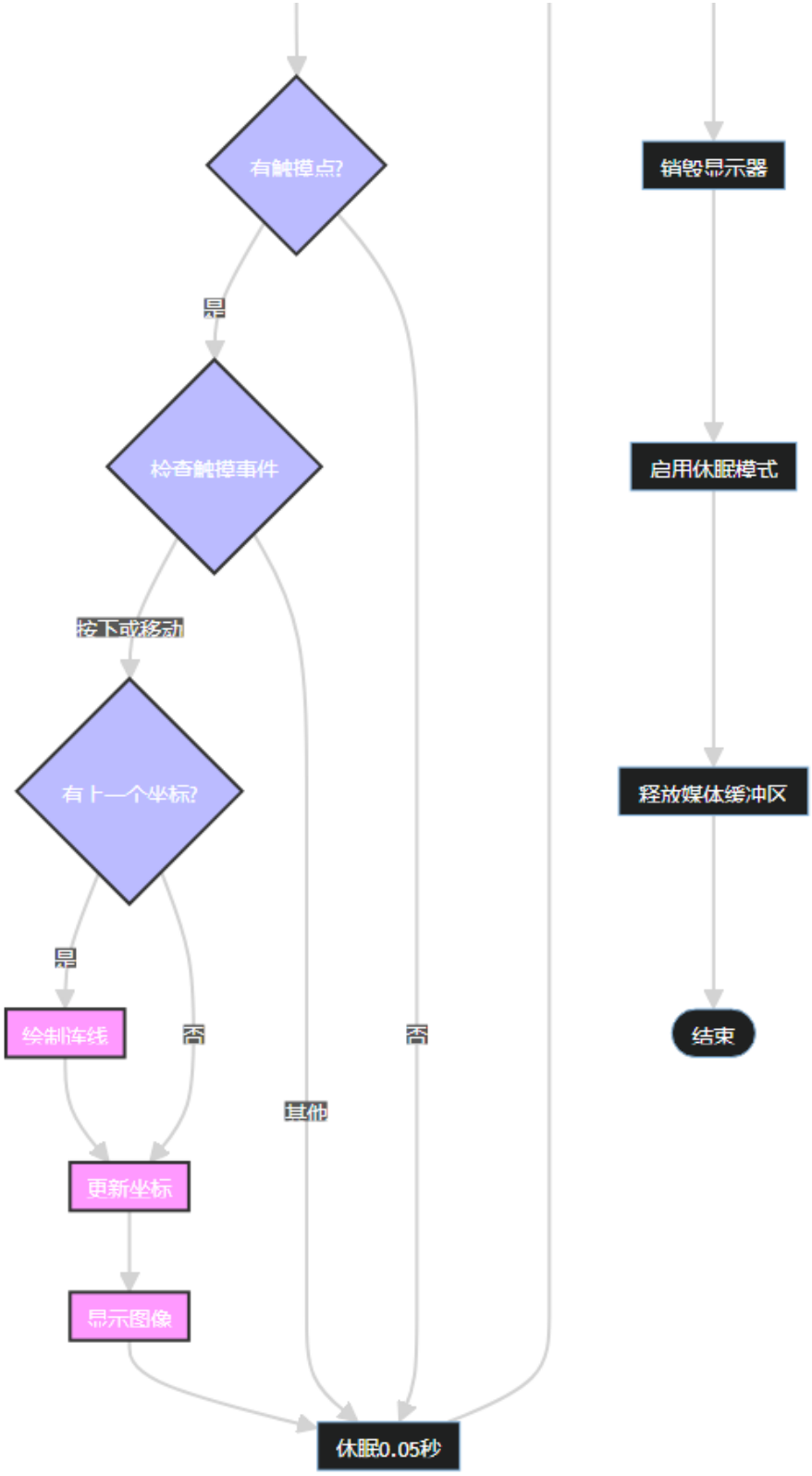
代码结构

1. 基础设置:
    - 导入必要模块
    - 初始化触摸设备
    - 定义显示常量
    - 创建和初始化图像
    - 初始化显示器和媒体管理器
2. 主循环中:
    - 检查是否需要退出
    - 读取触摸数据
    - 如果有触摸点:
        - 检查触摸事件类型
        - 如果有上一个坐标点,绘制连线
        - 更新坐标
        - 显示图像
    - 短暂休眠
3. 异常处理:
    - 处理键盘中断

    - 处理其他异常
4. 清理工作:
    - 销毁显示器
    - 启用休眠模式
    - 释放媒体缓冲区

完整流程图如下：


开始
导入必要的模块
初始化触摸设备
定义显示常量
定义display_test函数
创建绘图图像
清除图像
初始化显示器
初始化媒体管理器
try代码块
主循环
键盘中断
其他异常
检查是否退出
打印用户停止信息
打印异常信息
否
是
读取触摸数据
清理工作



TOUCH 相关文档

> 以下内容选自CanMV K230 TOUCH API
>
> 2.16 TOUCH 模块 API 手册 — CanMV K230

1. 概述

触摸模块基于 RTT 的触摸框架，支持单点和多点电容触摸屏及电阻触摸屏。

2. API 介绍

TOUCH 类位于 `machine` 模块下。

示例

```
from machine import TOUCH

# 实例化 TOUCH 设备 0
tp = TOUCH(0)
# 获取 TOUCH 数据
p = tp.read()
print(p)
# 打印触摸点坐标
# print(p[0].x)
# print(p[0].y)
# print(p[0].event)
```

构造函数

```
# when index is 0
touch = TOUCH(index, type = TOUCH.TYPE_CST328, rotate = -1)

# when index is 1
touch = TOUCH(index, type = TOUCH.TYPE_CST328, rotate = -1, range_x = -1, range_y
= -1, i2c : I2C = None, slave_addr = None, rst : Pin = None, int : Pin = None)
```

参数

- `index`：`TOUCH` 设备号，为 `0` 时，表示使用系统自带的触摸，为 `1` 时，表示使用 `CanMV` 专有的触摸驱动
- `type`：触摸驱动类型，具体定义参考触摸类型
- `rotate`：面板输出坐标与屏幕坐标的旋转，取值范围为 [0-3]，具体定义参考坐标旋转。
- `range_x`：`index=1` 时有效，触摸输出坐标的宽度最大值
- `range_y`：`index=1` 时有效，触摸输出坐标的高度最大值
- `i2c`：`index=1` 时有效，触摸使用 `I2C` 总线对象
- `slave_addr`：`index=1` 时有效，触摸芯片的从机地址，可选参数，不传入使用驱动默认值
- `rst`：`index=1` 时有效，触摸复位引脚对象
- `int`：`index=1` 时有效，触摸中断引脚对象，当前不支持

`read` 方法

```
TOUCH.read([count])
```

获取触摸数据。

参数

- `count`：最多读取的触摸点数量，取值范围为 [0:10]，默认为 0，表示读取所有触摸点。

返回值

返回触摸点数据，类型为元组 `([tp[, tp...]])`，其中每个 `tp` 是一个 `touch_info` 类实例。

`deinit` 方法

```
TOUCH.deinit()
```

释放 TOUCH 资源。

参数

无

返回值

无

3. TOUCH_INFO 类

TOUCH_INFO 类用于存储触摸点的信息，用户可通过相关只读属性访问。

- `event`：事件码，具体参考触摸事件。
- `track_id`：触点 ID，用于多点触摸。
- `width`：触点宽度。
- `x`：触点的 x 坐标。
- `y`：触点的 y 坐标。
- `timestamp`：触点时间戳。

4. 常量

4.1 触摸事件

- `EVENT_NONE`：无事件。
- `EVENT_UP`：触摸按下后抬起。
- `EVENT_DOWN`：触摸按下开始。
- `EVENT_MOVE`：触摸按下后移动。

4.2 坐标旋转

- `ROTATE_0`：坐标不旋转。
- `ROTATE_90`：坐标旋转 90 度。
- `ROTATE_180`：坐标旋转 180 度。
- `ROTATE_270`：坐标旋转 270 度。

4.3 触摸类型

- `TYPE_CST328`：`CanMV` 专有触摸驱动
- `TYPE_CST226SE`：`CanMV` 专有触摸驱动
- `TYPE_GT911`：`CanMV` 专有触摸驱动